

Chat Widget Message Flow

DistributedOrderSystem — end-to-end trace of a customer chat message, from the browser widget through GatewayBff, ChatbotService, and the AgentService LangGraph agent, back to the customer · generated 2026-09-22

1. Overview

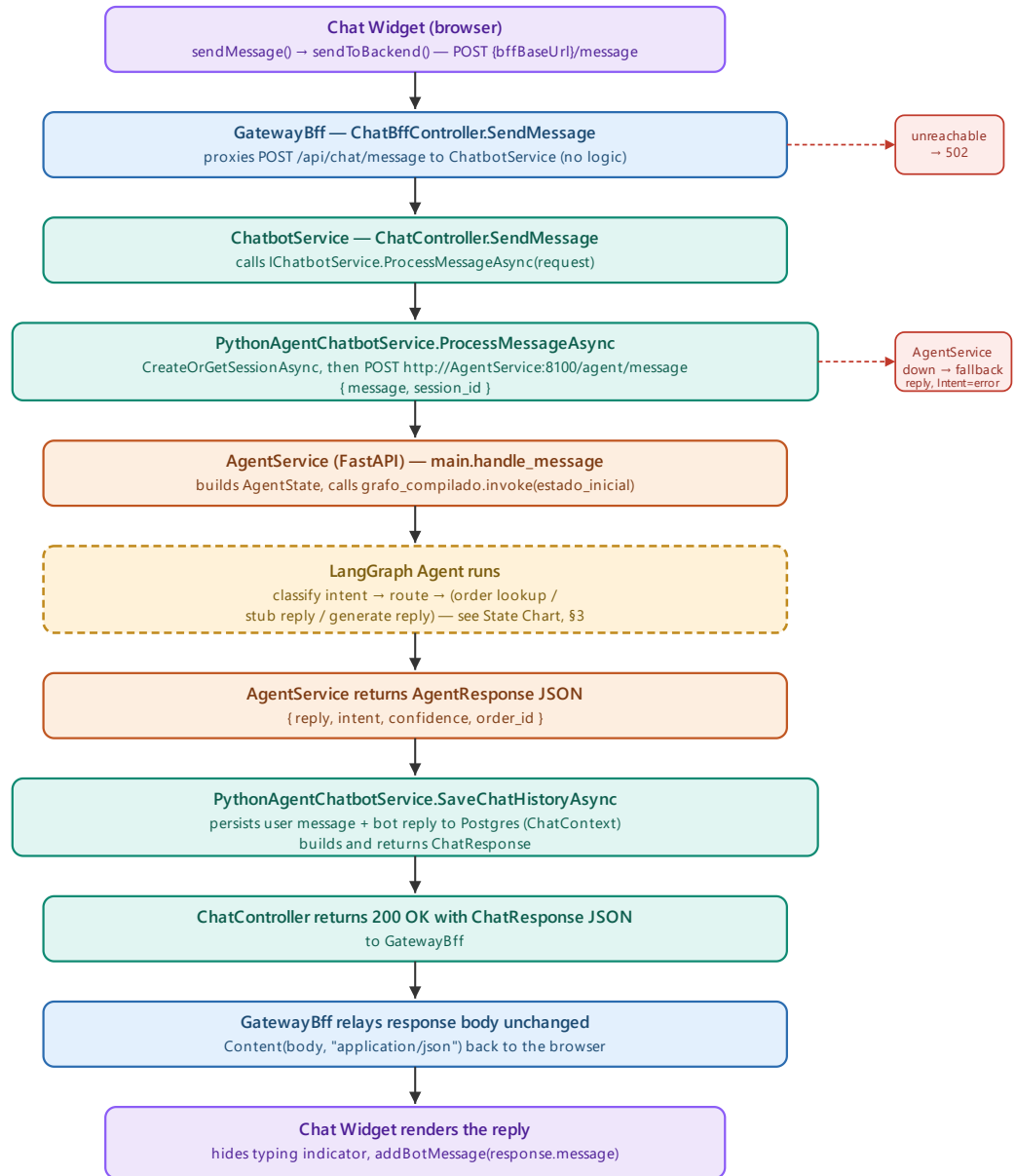
The chat widget embedded in the customer-facing site sends every message to `GatewayBff`, which proxies it to `ChatbotService`. `ChatbotService` does not run any AI logic itself — it forwards the message over HTTP to a separate Python process, `AgentService`, which runs a LangGraph agent backed by Claude. The agent classifies the customer's intent, optionally looks up a real order by calling back into `GatewayBff`, generates a natural-language reply, and the response travels back up the same chain to the browser.

Three diagrams follow, each answering a different question:

- **Flowchart (§2)** — what happens, step by step, across all services for one message.
- **State chart (§3)** — the internal states of the LangGraph agent inside `AgentService`.
- **Component diagram (§4)** — the static architecture: which service talks to which, and how.

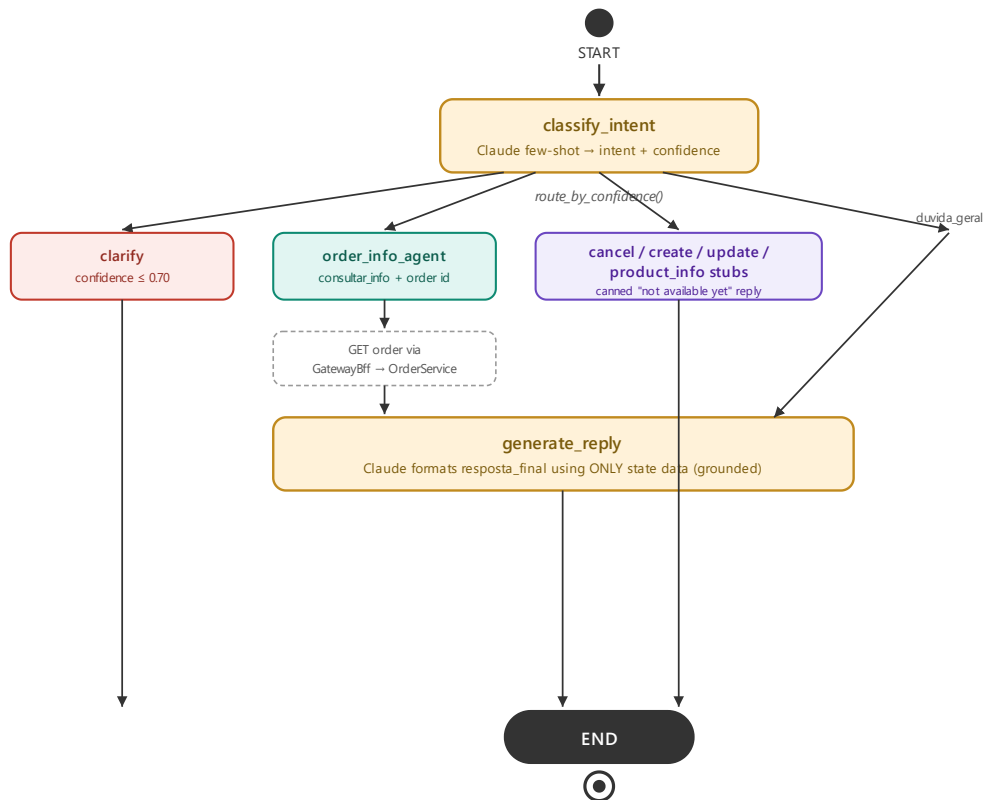
Section 5 lists every file involved in call order with exact line numbers, and where to set breakpoints to debug the round trip in both the .NET and Python debuggers at once.

2. Flowchart — request/response lifecycle



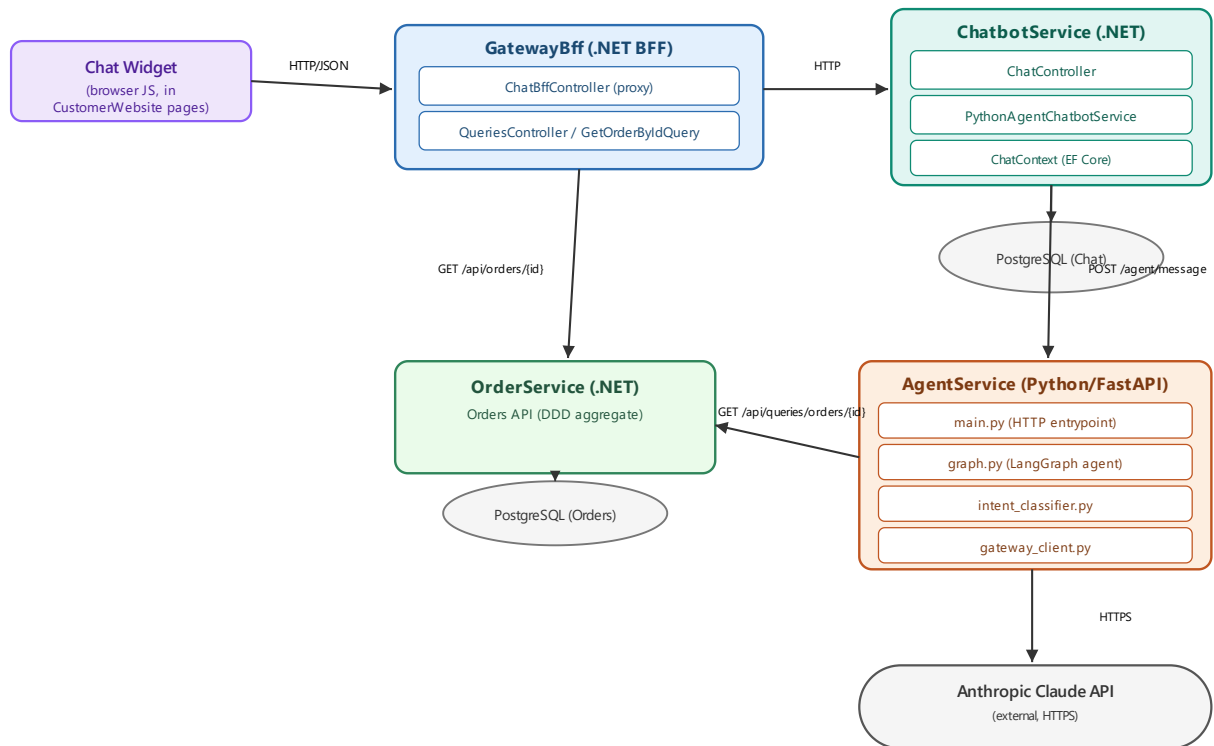
3. State chart — LangGraph agent (inside AgentService)

This is the state machine defined in `graph.py`. `classify_intent` is the only entry point; `route_by_confidence()` is a pure function (no model call) that decides which branch runs next based on the classifier's confidence score and intent label.



Every branch is a leaf that returns straight to ChatbotService via END — only order_info_agent and duvida_geral pass through generate_reply.

4. Component diagram — static architecture



AgentService is the only component that calls Claude directly; it also re-enters GatewayBff (a second, independent HTTP call) purely to read order data.

5. File & breakpoint reference

Call order for a single message (numbers match the flowchart steps in §2):

#	Layer	File	Key method / line
1	Browser	ChatbotService/wwwroot/chat-widget/dist/chat-widget.min.js	sendMessage() L536, sendToBackend() L568
2	GatewayBff	GatewayBff/Controllers/ChatBffController.cs	SendMessage() L73
3	ChatbotService	ChatbotService/Controllers/ChatController.cs	SendMessage() L33
4	ChatbotService	ChatbotService/Services/PythonAgentChatbotService.cs	ProcessMessageAsync() L36, HTTP call L43
5	AgentService	AgentService/main.py	handle_message() L35
6	AgentService	AgentService/intent_classifier.py	classify_intent_node() L62
7	AgentService	AgentService/graph.py	route_by_confidence() L80, order_info_agent_node() L22, generate_reply_node() L50
8	AgentService	AgentService/gateway_client.py	get_order_by_id() L15
9	GatewayBff	GatewayBff/Queries/GetOrderByIdQuery.cs	Handle() L21
10	ChatbotService	ChatbotService/Services/PythonAgentChatbotService.cs	SaveChatHistoryAsync() L95

Recommended breakpoints — .NET side (launch the `Start Agent + Chatbot + Gateway` compound, or the full `Start All (Backend + Frontend)`):

File : line	Why stop here
ChatBffController.cs : 73	Confirms the widget request reached the BFF; inspect the raw request body.
ChatBffController.cs : 80	Right before forwarding to ChatbotService — catch here explains a 502.
ChatController.cs : 42	Entry into business logic; inspect request.Message / request.SessionId.
PythonAgentChatbotService.cs : 38	Session creation / lookup.
PythonAgentChatbotService.cs : 43	Outbound call to AgentService — step over to see the full round trip.
PythonAgentChatbotService.cs : 53	Catch block — tells you immediately if AgentService is unreachable.
GetOrderByIdQuery.cs : 21	Only hit when the agent asks about a specific order.

Recommended breakpoints — Python side (needs the `AgentService (Python)` debugpy launch config attached — a .NET breakpoint alone will not stop Python code):

File : line	Why stop here
main.py : 35	First line inside AgentService — confirms the HTTP call from ChatbotService landed.
intent_classifier.py : 69	Just before the Claude call that classifies intent.
graph.py : 80	route_by_confidence — pure function, instant, cleanest place to watch routing decisions.
graph.py : 26	order_info_agent_node — only hit for order-info intents.
gateway_client.py : 23	Outbound call to GatewayBff for the order lookup.
graph.py : 68	generate_reply_node — right before the final Claude call.

Debugging tip: the ChatbotService → AgentService hop is a real HTTP call, not an in-process call, so a .NET breakpoint cannot step into the Python code. Launch AgentService (Python) and ChatbotService together (or use the Start Agent + Chatbot + Gateway compound), set breakpoints on both sides, then send a message from the widget. VS Code stops at whichever breakpoint is hit first; switch between the two debug sessions from the Call Stack panel.

Cost note: intent_classifier.py:69 and graph.py:68 make real calls to the Anthropic API (claude-sonnet-4-6). Leaving a breakpoint there just delays the response — it doesn't break anything — but each debug run still costs real tokens.